

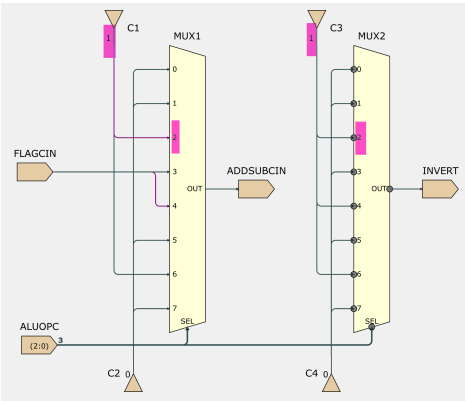
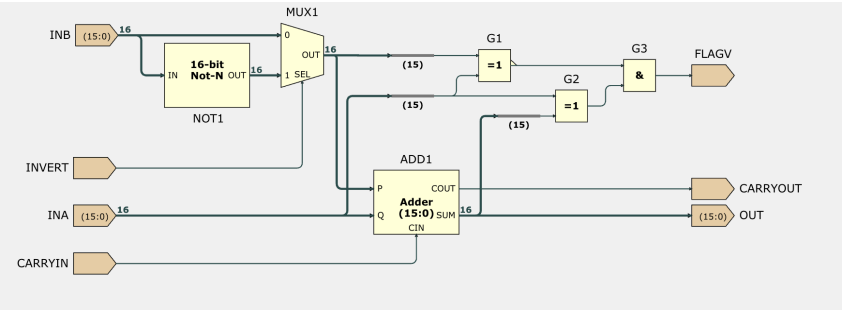
Section 5

2026年1月28日 星期三 10:20

5 ADDSUB Block design in the ALU

Task 5. The five arithmetic instructions ADD, SUB, ADC, SBC, and CMP all use the ADDSUB and ALU.DECODE blocks.

1. Function of ADDSUB. Explain the function of the ADDSUB block in terms of its inputs. A strong way to support your explanation is to write an algebraic truth table for ADDSUB.OUT as a function of the algebraic inputs INA, INB, CARRYIN and the binary control input INVERT. Express the behaviour symbolically (for example, "OUT = INA + INB + CARRYIN" when INVERT = 0, etc.).



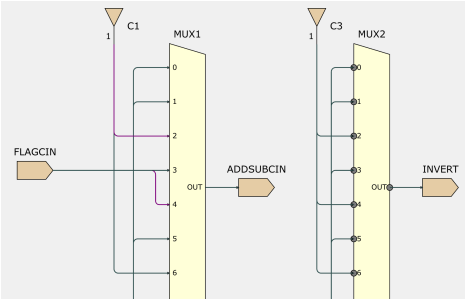
A	B	C
INVERT	CARRYIN	OUT
0	0	0 INA+INB+CARRYIN
1	1	1 INA+INB+CARRYIN

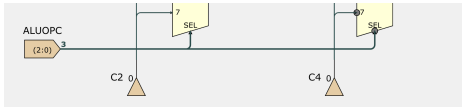
ADD1 just add P&Q normally together. INVERT controls whether to take 2's compliment of INB, when INVERT=1, all INB bits negate, and there's another +1 in CARRYIN, so INB takes its 2's compliment, the result of P&Q becomes subtraction.

Seen from ALUDECODE, in 2, subtraction, ADDSUBCIN, which connects to CARRYIN of ADDSUB = 1. INVERT also = 1.

2. Using your answer to the previous question and the specification of the five arithmetic instructions in Figure 8 of the handout, explain how the control signals from ALU.DECODE into ADDSUB allow all five instructions (ADD, SUB, ADC, SBC, and CMP) to be executed. The clearest way to do this is to write an algebraic truth table for the ALU.DECODE block, with outputs ALU.DECODE.INVERT and ALU.DECODE.ADDSUBCIN, and inputs ALU.DECODE.ALUOPC and ALU.DECODE.FLAGCIN.

A	B	C	D	E
ALUOPC	Instructions	FLAGCIN	INVERT	ADDSUBCIN
1	ADD	X	0	0
3	ADC	C	0	C
2	SUB	X	1	1
4	SBC	C	1	C
6	CMP	X	1	1





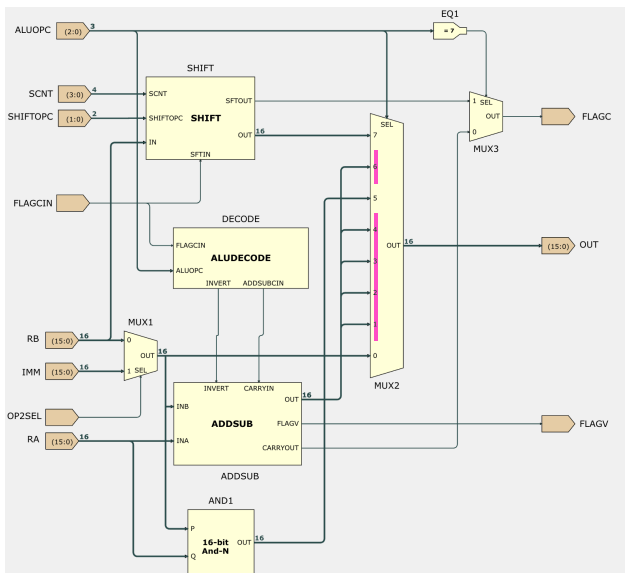
Truth table shows value read from logic gates, and it aligns well with the instructions, where INVERT is just about inverting number. ADDSUBCIN has 2 functions, takes carry-in FLAGCIN, and 2's complement.

Notice: in subtraction, FLAGCIN = 1 stands for no carry-in.

For instructions other than the five considered here, ADDSUB.OUT is a don't-care. By looking at where the wire from ADDSUB.OUT goes, explain why its value does not affect the correct behaviour of the CPU for those instructions.

ALUOPC	INS(8)=0	INS(8)=1
0	MOV ¹ Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP ¹ Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

¹Instruction does not use Rc, this field is (0)



These 5 instructions has ALUOPC 12346, so when ALUOPC = 057, MUX2 won't choose ADDSUB.OUT.

3. Explain how the hardware implementation of ALU.DECODE achieves the behaviour you have described in your truth table. Focus on how the combination of ALUOPC and FLAGCIN is mapped onto the control signals INVERT and ADDSUBCIN.

Described in 1.

1. What is the purpose of the FLAGCIN input? Which type of operation does it enable, and for which instructions is it used?

FLAGCIN allows the carry flag from a previous operation to be used in the current operation, enabling ADC and SBC instructions and supporting multi-precision (with more bits) arithmetic through carry/borrow propagation.